fuaddary tugas week 12 6cc4216 · 5 days ago

527 lines (391 loc) · 21.5 KB

Tugas Lanjutan ETS: Upgrade Pipeline ke Data Lakehouse

Mata Kuliah	Big Data dan Data Lakehouse
Jenis Tugas	Tugas Kelompok Lanjutan ETS (Kelompok sama dengan ETS)
Pertemuan Terkait	P12 (Data Lakehouse), lanjutan ETS (Kafka + HDFS + Spark)
Deadline	Dikerjakan Week 12, dipresentasikan Week 13
Deliverables	Tambahan kode di repo ETS (folder `lakehouse/`) + presentasi 10 menit

🎯 Konteks: Apa yang Sudah Kalian Bangun?

Pada ETS, kelompok kalian membangun pipeline Big Data end-to-end:

```
[API Real-time] → Kafka → Consumer → HDFS (/data/[tema]/api/)
[RSS Feed]      → Kafka → Consumer → HDFS (/data/[tema]/rss/)
                                        ↓
                                  Spark analysis.py
```

```
                    (3 analisis, simpan ke HDFS)
                          ↓
                    Flask Dashboard
```

Data sudah tersimpan di HDFS sebagai **file JSON mentah** — tidak ada transaksi ACID, tidak ada versioning, tidak ada schema yang ketat. Jika ada update data, tidak ada cara mudah untuk melacak perubahannya.

🎯 Apa yang Harus Ditambahkan di Tugas Ini?

Kalian akan **meng-upgrade pipeline ETS** dengan menambahkan lapisan **Data Lakehouse (Medallion Architecture + Delta Lake)** di atas data yang sudah ada di HDFS.

```
SEBELUM (ETS):                          SESUDAH (Tugas Ini):

[API/RSS] → Kafka → HDFS (JSON) → Spark   [API/RSS] → Kafka → HDFS (JSON)
                                                              ↓
                                                  [BRONZE] Delta Lake (raw)
                                                              ↓
                                                  [SILVER] Delta Lake (cleaned)
                                                              ↓
                                                   [GOLD] Delta Lake (aggregated)
                                                              ↓
                                                   Dashboard (baca dari Gold)
```

Perbedaan utama yang harus kalian rasakan:

- Sebelumnya: Spark membaca JSON mentah dari HDFS — tidak ada schema enforcement, tidak ada versioning
- Sesudah: Data tersimpan di Delta Lake dengan ACID, bisa di-query ulang di versi mana pun

📋 Yang Harus Dibangun

Tambahkan folder `lakehouse/` di dalam repository ETS kalian dengan struktur berikut:

```
[repo-ets-kalian]/
├── ... (kode ETS yang lama, jangan diubah)
└── lakehouse/
    ├── README_lakehouse.md     ← Dokumentasi khusus tugas ini
    ├── 00_setup.md             ← Cara menjalankan Spark + Delta Lake
```

```
├── 01_bronze.py            ← Ingest dari HDFS ke Bronze Delta layer
├── 02_silver.py            ← Cleaning → Silver Delta layer
└── 03_gold.py              ← Aggregasi → Gold Delta layer
```

🔧 Spesifikasi Teknis (3 Script Utama)

Script 1: `01_bronze.py` — Ingest dari HDFS ke Bronze Layer

Sumber data: File JSON yang sudah ada di HDFS dari consumer ETS kalian:

- `hdfs://namenode:8020/data/[tema]/api/`
- `hdfs://namenode:8020/data/[tema]/rss/`

Yang harus dilakukan:

1. Baca semua file JSON dari HDFS menggunakan PySpark (`spark.read.json(...)`)
2. Tambahkan kolom metadata: `_ingested_at` (waktu ingest sekarang) dan `_source` (nama sumber, misal `"api"` atau `"rss"`)
3. Simpan ke format **Delta Lake** (bukan parquet biasa)

```
# Hint: Cara inisialisasi SparkSession dengan Delta Lake + HDFS
from pyspark.sql import SparkSession
from delta import configure_spark_with_delta_pip
from pyspark.sql.functions import current_timestamp, lit

builder = SparkSession.builder.appName("Bronze-[Tema]") \
    .config("spark.sql.extensions", "io.delta.sql.DeltaSparkSessionExtension") \
    .config("spark.sql.catalog.spark_catalog", "org.apache.spark.sql.delta.catalog
    .config("spark.hadoop.fs.defaultFS", "hdfs://namenode:8020")

spark = configure_spark_with_delta_pip(
    builder, extra_packages=["io.delta:delta-spark_2.12:3.1.0"]
).getOrCreate()

# Baca dari HDFS (path sesuai tema ETS kalian)
api_df = spark.read.option("multiLine", True).json("hdfs://namenode:8020/data/[tem

# Tambahkan metadata
bronze_df = api_df.withColumn("_ingested_at", current_timestamp()) \
                  .withColumn("_source", lit("api"))
```

```
# Simpan ke Delta Lake (Bronze) — bisa di lokal atau HDFS
bronze_df.write.format("delta").mode("append").save("./lakehouse_data/bronze/[tema
```

> 📌 **Catatan**: Jika Hadoop tidak aktif, boleh menggunakan file JSON lokal yang sudah didownload dari HDFS sebagai alternatif. Dokumentasikan pilihan ini.

Script 2: `02_silver.py` — Cleaning ke Silver Layer

Sumber data: Bronze Delta layer yang dibuat di script 1

Yang harus dilakukan:

1. Baca dari Bronze Delta layer
2. Lakukan **minimal 3 transformasi cleaning** yang relevan dengan domain ETS kalian:

Jenis Transformasi	Contoh untuk Topik Berbeda
Hapus duplikat	`dropDuplicates(["id_unik_data"])`
Cast tipe data	String timestamp → `to_timestamp()`
Handle null	`fillna()` atau `filter(col(...).isNotNull())`
Filter data invalid	Harga < 0, magnitude < 0, AQI < 0
Ekstrak kolom	Ambil jam dari timestamp (`hour(col("timestamp"))`)
Standarisasi nilai	Uppercase nama kota, dsb.

3. Simpan ke Silver Delta layer

```
# Contoh cleaning (sesuaikan dengan skema domain kalian)
from pyspark.sql.functions import col, to_timestamp, hour

bronze_df = spark.read.format("delta").load("./lakehouse_data/bronze/[tema]_api")

silver_df = bronze_df \
    .dropDuplicates(["[kolom_id_unik]"]) \
    .filter(col("[kolom_nilai_utama]").isNotNull()) \
    .withColumn("timestamp", to_timestamp(col("timestamp"))) \
    .withColumn("jam", hour(col("timestamp")))

silver_df.write.format("delta").mode("overwrite").save("./lakehouse_data/silver/[t
```

> 💡 **Justifikasi Transformasi**: Di file `README_lakehouse.md`, jelaskan MENGAPA setiap transformasi dilakukan. Berapa baris data yang hilang setelah cleaning? Apa artinya?

Script 3: `03_gold.py` — Agregasi & Enhanced Analysis di Gold Layer

Sumber data: Silver Delta layer (API + RSS)

Mengapa Medallion Memungkinkan Analisis Lebih Baik?

Di ETS, Spark membaca JSON mentah dari HDFS — tipe data belum tentu benar, ada duplikat, timestamp belum di-parse. Akibatnya analisis terbatas pada agregasi dasar.

Dengan Silver yang sudah bersih:

- Timestamp sudah bertipe `TimestampType` → bisa pakai **Window Functions** (lag, rolling avg)
- Tidak ada duplikat → agregasi akurat
- Kolom `jam` sudah diekstrak → analisis temporal lebih mudah
- Data API + RSS bisa di-**join** → analisis lintas sumber

```
Silver (API)  ──┐
                ├── JOIN di Gold → Insight lintas sumber
Silver (RSS)  ──┘
```

Yang harus dilakukan: Buat **minimal 3 tabel Gold** — 2 tabel mereproduksi analisis ETS yang lama, dan **1 tabel Enhanced** yang tidak bisa dibuat di ETS.

🗂️ Panduan Gold Enhanced per Topik ETS

Pilih bagian yang sesuai topik kelompok kalian:

💰 Topik 1 — CryptoWatch

Tabel Gold	ETS?	Enhancement
`gold/crypto_stats`	✅ Repro	avg/min/max/stddev per simbol
`gold/crypto_hourly_volatility`	✅ Repro	avg |change_24h| per jam

Tabel Gold	ETS?	Enhancement
gold/crypto_spike_alerts	🆕 Enhanced	Deteksi anomali harga (event di mana harga menyimpang > 2× stddev dari rata-rata)
gold/crypto_news_price_join	🆕 Enhanced	Join RSS + API: berapa artikel berita muncul dalam 1 jam sebelum/sesudah lonjakan harga?

```
from pyspark.sql.functions import avg, stddev, abs as spark_abs, col

silver_api = spark.read.format("delta").load("./lakehouse_data/silver/crypto_api")

# Hitung baseline statistik per simbol
baseline = silver_api.groupBy("symbol").agg(
    avg("price_usd").alias("mean_price"),
    stddev("price_usd").alias("std_price")
)

# Deteksi spike: harga > mean + 2*std
spike_df = silver_api.join(baseline, "symbol") \
    .withColumn("z_score", (col("price_usd") - col("mean_price")) / col("std_price
    .filter(spark_abs(col("z_score")) > 2) \
    .select("symbol", "price_usd", "timestamp", "z_score", "mean_price")

spike_df.write.format("delta").mode("overwrite").save("./lakehouse_data/gold/crypt
print(f"Ditemukan {spike_df.count()} kejadian lonjakan harga anomali")
```

🌤️ Topik 2 — WeatherPulse

Tabel Gold	ETS?	Enhancement
gold/weather_city_stats	✅ Repro	avg/max/min suhu per kota
gold/weather_extreme_events	✅ Repro	Hitungan kondisi ekstrem per kota
gold/weather_comfort_index	🆕 Enhanced	Heat Index = f(suhu, kelembaban) — rasanya seperti berapa derajat?
gold/weather_alert_summary	🆕 Enhanced	Join RSS: berapa berita cuaca muncul saat ada kondisi ekstrem?

```python
from pyspark.sql.functions import col, expr

silver_api = spark.read.format("delta").load("./lakehouse_data/silver/weather_api"

# Heat Index = -8.78 + 1.61*T + 2.34*RH - 0.146*T*RH (formula sederhana)
gold_comfort = silver_api.withColumn(
    "heat_index",
    expr("-8.78 + 1.61*temperature + 2.34*humidity - 0.146*temperature*humidity")
).withColumn(
    "feels_like_category",
    expr("""
        CASE
            WHEN heat_index < 27 THEN 'Nyaman'
            WHEN heat_index < 32 THEN 'Perlu Hati-hati'
            WHEN heat_index < 41 THEN 'Berbahaya'
            ELSE 'Sangat Berbahaya'
        END
    """)
).groupBy("kode_kota", "feels_like_category").count() \
 .orderBy("kode_kota", "count", ascending=False)

gold_comfort.write.format("delta").mode("overwrite").save("./lakehouse_data/gold/w
```

Topik 3 — AirQuality Alert

Tabel Gold	ETS?	Enhancement
`gold/aqi_category_dist`	✅ Repro	Distribusi kategori AQI per kota
`gold/aqi_hourly`	✅ Repro	Rata-rata AQI per jam
`gold/aqi_trend`	NEW **Enhanced**	Tren AQI: apakah kualitas udara membaik atau memburuk dari waktu ke waktu? (lag comparison)
`gold/aqi_alert_hours`	NEW **Enhanced**	Berapa jam berturut-turut setiap kota dalam kondisi "Tidak Sehat" (AQI > 150)?

```python
from pyspark.sql import Window
```

```
window_spec = Window.partitionBy("kota").orderBy("timestamp")

gold_trend = silver \
    .withColumn("prev_aqi", lag("aqi", 1).over(window_spec)) \
    .withColumn("aqi_change", col("aqi") - col("prev_aqi")) \
    .withColumn("trend", when(col("aqi_change") > 5, "Memburuk")
                         .when(col("aqi_change") < -5, "Membaik")
                         .otherwise("Stabil")) \
    .groupBy("kota", "trend").count()

gold_trend.write.format("delta").mode("overwrite").save("./lakehouse_data/gold/aqi
```

📈 Topik 4 — SahamMeter

Tabel Gold	ETS?	Enhancement
`gold/saham_return`	✅ Repro	Return % per saham
`gold/saham_volatility`	✅ Repro	Stddev harga per saham
`gold/saham_sharpe_proxy`	🆕 **Enhanced**	Sharpe Proxy = avg_return / stddev (risk-adjusted performance)
`gold/saham_news_mention`	🆕 **Enhanced**	Join RSS: frekuensi sebutan nama perusahaan per jam, korelasi dengan pergerakan harga

```
from pyspark.sql.functions import avg, stddev, col

silver = spark.read.format("delta").load("./lakehouse_data/silver/saham")

stats = silver.groupBy("ticker").agg(
    avg("return_pct").alias("avg_return"),
    stddev("return_pct").alias("risk")
)

gold_sharpe = stats.withColumn(
    "sharpe_proxy",
    col("avg_return") / col("risk")
).orderBy("sharpe_proxy", ascending=False)

gold_sharpe.show()
gold_sharpe.write.format("delta").mode("overwrite").save("./lakehouse_data/gold/sal
```

🌍 Topik 5 — NewsPulse

Tabel Gold	ETS?	Enhancement
`gold/word_freq`	✅ Repro	Frekuensi kata per judul
`gold/news_per_source`	✅ Repro	Volume berita per sumber
`gold/word_velocity`	🆕 **Enhanced**	Kata yang frekuensinya paling cepat naik dalam 2 jam terakhir (trending)
`gold/cross_source_topics`	🆕 **Enhanced**	Apakah topik yang sama muncul di API dan RSS dalam waktu berdekatan?

```
from pyspark.sql.functions import split, explode, col, lower, hour, count

silver = spark.read.format("delta").load("./lakehouse_data/silver/news")

# Word velocity: hitung frekuensi kata per jam, lihat perubahan antar jam
words_per_hour = silver \
    .withColumn("word", explode(split(lower(col("judul")), " "))) \
    .filter(col("word").rlike("^[a-z]{4,}")) \
    .groupBy("jam", "word").count()

words_per_hour.write.format("delta").mode("overwrite").save("./lakehouse_data/gold
print("Top 10 kata per jam:")
words_per_hour.orderBy("jam", "count", ascending=[True, False]).show(10)
```

🌋 Topik 6 — GempaRadar

Tabel Gold	ETS?	Enhancement
`gold/gempa_mag_dist`	✅ Repro	Distribusi kategori magnitudo
`gold/gempa_region_rank`	✅ Repro	Wilayah paling aktif
`gold/gempa_risk_score`	🆕 **Enhanced**	Risk Score per wilayah = frekuensi × rata-rata magnitudo × (1 + % gempa dangkal)
`gold/gempa_significant_alerts`	🆕 **Enhanced**	Join RSS: gempa M>4.5 dan berita kebencanaan yang muncul dalam 2 jam sesudahnya

```python
from pyspark.sql.functions import count, avg, col, when

silver = spark.read.format("delta").load("./lakehouse_data/silver/gempa")

gold_risk = silver.groupBy("wilayah").agg(
    count("*").alias("frekuensi"),
    avg("magnitude").alias("avg_mag"),
    (count(when(col("depth") < 70, True)) / count("*")).alias("pct_dangkal")
).withColumn(
    "risk_score",
    col("frekuensi") * col("avg_mag") * (1 + col("pct_dangkal"))
).orderBy("risk_score", ascending=False)

gold_risk.write.format("delta").mode("overwrite").save("./lakehouse_data/gold/gempa
print("Top 10 wilayah berisiko tinggi:")
gold_risk.show(10)
```

💻 Topik 7 — GitTrend

Tabel Gold	ETS?	Enhancement
`gold/language_dist`	✅ Repro	Distribusi bahasa pemrograman
`gold/top_repos`	✅ Repro	Top 10 repo berdasarkan bintang
`gold/star_velocity`	🆕 **Enhanced**	Star velocity per repo: perubahan jumlah bintang antar observasi (deteksi repo yang sedang viral)
`gold/emerging_topics`	🆕 **Enhanced**	Kata kunci di deskripsi yang muncul dalam 3 jam terakhir tapi belum ada sebelumnya

```python
from pyspark.sql import Window
from pyspark.sql.functions import lag, col

silver = spark.read.format("delta").load("./lakehouse_data/silver/github")

# Star velocity: perubahan bintang antar observasi
window_spec = Window.partitionBy("full_name").orderBy("timestamp")

gold_velocity = silver \
    .withColumn("prev_stars", lag("stargazers_count", 1).over(window_spec)) \
    .withColumn("star_gain", col("stargazers_count") - col("prev_stars")) \
    .groupBy("full_name", "language") \
    .agg({"star_gain": "sum", "stargazers_count": "max"}) \
```

```
    .orderBy("sum(star_gain)", ascending=False)

gold_velocity.write.format("delta").mode("overwrite").save("./lakehouse_data/gold/
print("Repo yang paling banyak mendapat bintang baru:")
gold_velocity.show(10)
```

🛒 Topik 8 — HargaPangan

Tabel Gold	ETS?	Enhancement
`gold/pangan_volatility`	✅ Repro	Indeks volatilitas per komoditas
`gold/pangan_trend`	✅ Repro	Rata-rata harga per periode
`gold/pangan_alert`	🆕 **Enhanced**	Early warning: komoditas dengan kenaikan harga > 5% dalam 3 observasi terakhir
`gold/pangan_news_correlation`	🆕 **Enhanced**	Join RSS: komoditas yang sering disebut berita vs fluktuasi harga aktual

```
from pyspark.sql import Window
from pyspark.sql.functions import lag, col, when

silver = spark.read.format("delta").load("./lakehouse_data/silver/pangan")

# Early warning: deteksi kenaikan harga > 5% dari harga sebelumnya
window_spec = Window.partitionBy("komoditas").orderBy("timestamp")

gold_alert = silver \
    .withColumn("prev_harga", lag("harga", 1).over(window_spec)) \
    .withColumn("pct_change", (col("harga") - col("prev_harga")) / col("prev_harga
    .withColumn("alert", when(col("pct_change") > 5, "⚠️ NAIK SIGNIFIKAN")
                         .when(col("pct_change") < -5, "📉 TURUN SIGNIFIKAN")
                         .otherwise("Normal")) \
    .filter(col("alert") != "Normal") \
    .select("komoditas", "harga", "prev_harga", "pct_change", "alert", "timestamp"

gold_alert.write.format("delta").mode("overwrite").save("./lakehouse_data/gold/pan
print(f"{gold_alert.count()} kejadian fluktuasi harga signifikan terdeteksi")
gold_alert.show()
```

🔍 Demonstrasi Time Travel (Wajib Ada)

Setelah membuat Silver layer, tunjukkan kemampuan **Time Travel Delta Lake**:

```
from delta.tables import DeltaTable
from pyspark.sql.functions import lit

silver_path = "./lakehouse_data/silver/[tema]"
deltaTable = DeltaTable.forPath(spark, silver_path)

# 1. Lihat history tabel
print("=== History Tabel Silver ===")
deltaTable.history().select("version", "timestamp", "operation").show()

# 2. Lakukan sebuah perubahan (misalnya: update salah satu nilai)
# Contoh: update semua null jadi nilai default
deltaTable.update(
    condition="[kolom_tertentu] IS NULL",
    set={"[kolom_tertentu]": lit("[nilai_default]")}
)

# 3. Bandingkan data sebelum dan sesudah update
print("=== Data SEKARANG ===")
spark.read.format("delta").load(silver_path).groupBy("[kolom_tertentu]").count().sl

print("=== Data VERSI 0 (sebelum update) ===")
spark.read.format("delta").option("versionAsOf", 0).load(silver_path) \
    .groupBy("[kolom_tertentu]").count().show()
```

📝 Deliverables

1. Kode (Folder `lakehouse/` di repo ETS)

- [ ] `01_bronze.py` berjalan tanpa error
- [ ] `02_silver.py` berjalan, ada minimal 3 transformasi terdokumentasi
- [ ] `03_gold.py` berjalan, minimal 2 tabel Gold tersimpan di Delta format
- [ ] Demonstrasi Time Travel ada dan berjalan

2. `README_lakehouse.md`

Wajib berisi:

- Diagram arsitektur baru (sebelum vs sesudah ada Lakehouse)
- Penjelasan setiap transformasi di Silver (mengapa transformasi itu penting?)
- Perbandingan analisis Gold vs analisis Spark ETS yang lama (apa yang lebih baik?)
- Screenshot: output tabel Delta, hasil Time Travel
- Refleksi: **Apa keuntungan nyata Delta Lake dibanding menyimpan langsung di HDFS/CSV seperti sebelumnya?**

3. Presentasi Week 13 (10 menit per kelompok)

Durasi	Konten
2 menit	Demo arsitektur ETS lama vs baru (slide/diagram)
3 menit	Live demo: jalankan pipeline Bronze → Silver → Gold
2 menit	Demo Time Travel — tunjukkan data versi lama
3 menit	Insight dari Gold layer + tanya jawab

⚖️ Rubrik Penilaian

Total: 100 poin

Komponen	Skor Maks	Kriteria
Bronze Layer	15	Data dari HDFS berhasil diingest ke Delta format; metadata `_ingested_at` & `_source` ada; API + RSS keduanya masuk
Silver Layer	25	Minimal 3 transformasi cleaning relevan, terdokumentasi, jumlah baris berkurang dengan alasan valid
Gold — Reproduksi ETS	20	Minimal 2 tabel Gold yang mereproduksi analisis Spark ETS sebelumnya
Gold — Enhanced Analysis	20	Minimal 1 tabel Gold **Enhanced** (pakai Window Function, cross-source join, atau derived metric) yang tidak ada di ETS
Time Travel	10	Demonstrasi perubahan data (update/delete) dan query versi lama berhasil, output perbandingan ditampilkan

Komponen	Skor Maks	Kriteria
README & Refleksi	10	Diagram arsitektur sebelum/sesudah, justifikasi transformasi Silver, perbandingan hasil Gold vs ETS

Bonus (+10 poin):

- `+5` : Dashboard Flask diupdate membaca langsung dari tabel Gold Delta (bukan `spark_results.json`)
- `+3` : Cross-source join (gabungkan Silver API + Silver RSS) menghasilkan insight baru di Gold
- `+2` : Schema Evolution — tunjukkan penambahan kolom baru ke Silver menggunakan `mergeSchema`

❓ FAQ

Q: Data HDFS dari ETS sudah tidak ada / container sudah dihapus?

> Jalankan ulang `producer_api.py` dan `consumer_to_hdfs.py` selama ~10 menit untuk mengumpulkan data baru. Minimal 100 record sudah cukup untuk demonstrasi.

Q: Boleh menggunakan file JSON lokal (bukan dari HDFS)?

> Boleh, sebagai alternatif jika HDFS sulit diaktifkan. Catat keterbatasan ini di README. Nilai Bronze layer maks 15 poin.

Q: Apakah kode ETS yang lama harus diubah?

> **Tidak.** Tugas ini bersifat **additive** — hanya menambahkan folder `lakehouse/` baru. Kode ETS yang lama tidak boleh dimodifikasi agar perbandingan "sebelum vs sesudah" tetap jelas.

Q: Delta Lake disimpan di mana — HDFS atau lokal?

> Untuk kemudahan, simpan di folder lokal (`./lakehouse_data/`) di dalam container Spark. Jika mau disimpan ke HDFS sebagai bonus, nilai tidak ditambah tapi sangat diapresiasi.